Microsoft Purview Data Governance — Data Map, Scanning & Unified Catalog

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 5

Outcome . . . 6

1. Solution Overview — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog 6

What this solution is . . . 6

Why this matters . . . 6

The four building blocks . . . 6

Collections, governance domains, and roles . . . 7

Credentials and network — safe by design . . . 7

Licensing and the evolving portal . . . 7

What this is not . . . 7

The deployment model . . . 8

2. Requirements Guide — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog 8

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription . . . 8

2.2 Least-privilege data-governance roles (collections + Unified Catalog) . . . 8

2.3 Data sources in scope, inventoried with owners . . . 9

2.4 Source credentials via managed identity / Key Vault — never in code . . . 9

2.5 Network path to each source (integration runtime) . . . 9

2.6 A classification approach (system + any custom classifications) . . . 9

2.7 Governance-domain and data-product intent (Unified Catalog) . . . 9

2.8 A pilot scope and change control . . . 10

Checklist . . . 10

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 11

3. Security & Compliance PowerShell + Graph toolchain . . . 11

4. Data sources & scope in scope . . . 11

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 12

3. Implementation Guide — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog 12

Step 1 — Assess current state (read-only) . . . 13
Step 2 — Design the collection, domain, and role model . . . 13
Step 3 — Establish credentials and network paths (safe by design) . . . 13
Step 4 — Register the pilot sources into the Data Map . . . 13
Step 5 — Configure and pilot the scan (bounded scope, pilot-first) . . . 14
Step 6 — Review scanned assets and classifications . . . 14
Step 7 — Curate the Unified Catalog (glossary, data product, ownership) . . . 14
Step 8 — Wire lineage . . . 15
Step 9 — Establish data quality (verify availability) . . . 15
Step 10 — Validate against the pilot gate . . . 15
Step 11 — Expand across the estate (enforce) . . . 15
5. Testing Guide — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog 16
Test 1 — Account and roles are ready . . . 16
Test 2 — Pilot sources are registered and reachable . . . 16
Test 3 — The scan succeeds and returns assets . . . 16
Test 4 — Assets are classified . . . 16
Test 5 — The business glossary exists . . . 16
Test 6 — The data product is published with an owner . . . 16
Test 7 — Lineage resolves . . . 16
Test 8 — Data quality baseline (if available) . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog 17
A. Cannot reach the account / token fails . . . 17
B. Source registration or Test connection fails . . . 18
C. Scan runs but classifies nothing . . . 18
D. Classifications look wrong . . . 19
E. Unified Catalog curation issues . . . 19
F. Lineage is missing . . . 19
G. Data quality unavailable . . . 19
H. Diagnostics quick reference . . . 19
I. When to escalate . . . 20
7. Best Practices — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog 20
8. Deliverables Summary — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog 21
Documentation . . . 21

Named artifacts 21
PowerShell scripts 21
What the buyer receives 22
Outcome 22
Appendix A — PowerShell Scripts 23

Executive Summary — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog

Most data-governance efforts stop at Microsoft 365. But the data that carries the most risk — customer records in a database, financial extracts in a data lake, copies in a cloud bucket — lives *outside* M365, ungoverned and unclassified. You cannot protect, retain, or grant access to data you cannot see. This solution implements the **data-estate** side of Microsoft Purview: the **Data Map**, **scanning and classification**, and the **Unified Catalog** — so the whole estate becomes discoverable, classified, owned, and governed, not just Microsoft 365.

This is a complete, do-it-today package written for an engineer standing up Purview Data Governance for the first time. Every artifact is named once and reused everywhere, and every scan is proven on a bounded pilot before the estate is touched. Because the Purview Data Governance experience is evolving and its features are consumption-billed, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a path or a price as permanent.

What you get

• A plain-language Solution Overview of the four building blocks — the Data Map, scanning and

classification, the Unified Catalog (governance domains, data products, glossary, data quality), and lineage — and how they complement Microsoft 365 Purview.

• A Requirements Guide with verify / implement-if-missing for every prerequisite: the Purview account,

least-privilege collection and catalog roles, sources with owners, credentials via managed identity / Key Vault, the integration-runtime network path, a classification approach, and the pilot domain and data product.

• An Implementation Guide of precise, ordered steps: design the collection/domain/role model, establish

credentials and network safely, register and **pilot-scan** a bounded source set, review classifications, curate the catalog (glossary, data product, ownership), wire lineage and data quality, validate against the gate, then expand with scheduled scans.

• Five PowerShell tools for readiness/role discovery, source-registration and scan configuration

(`-WhatIf` by default), Unified Catalog curation (glossary / classification / data product), a scan-and-classification coverage and catalog-health report, and a pilot-scope helper — authentic Purview data-plane REST / Graph automation with **no credentials in code**.

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A governed data estate: sources mapped into the Data Map, assets scanned and classified, the Unified Catalog curated with governance domains, data products, a business glossary, and named owners, and lineage wired from source to report — validated on a pilot before you scale.

The deployment model

Assess → Design → Configure → **Pilot (bounded scan)** → Validate → Expand — with a pilot gate before any estate-wide scanning.

Outcome

The whole data estate discoverable, classified, owned, and governed — far beyond Microsoft 365 — with the business (not IT alone) able to find and trust data through the Unified Catalog, and regulatory, records, and access determinations left to the owners who make them.

1. Solution Overview — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog

What this solution is

Microsoft Purview has two related but distinct sides. The **compliance** side (labels, DLP, retention, insider risk) governs Microsoft 365 content. This solution implements the **data governance** side: the **Data Map**, **scanning and classification**, and the **Unified Catalog** — the experience that governs the *whole data estate*, not just Microsoft 365. That estate includes Azure storage and databases, multicloud sources (AWS, GCP), and on-premises systems. This is the evolution of what was formerly **Azure Purview / Microsoft Purview Data Catalog** — the portal, experience names, and billing model are **actively evolving**, so every capability below is flagged "verify your licensing/tenant" where availability or naming may have moved.

> ***Boundary.*** *This solution provides the technical method to *map, scan, classify, catalog, and wire lineage* across the data estate. It does **not** make the business or regulatory determinations — which data is "confidential", what the retention or records schedule is, or who may access what — those belong to the organization's data owners, stewards, compliance, privacy, and legal functions. The solution gives them a governed, discoverable estate to make those decisions against.*

Why this matters

Beyond Microsoft 365, most organizations have no idea where their sensitive data actually lives. Customer records sit in an Azure SQL database, financial extracts land in a data lake, a copy of a table gets exported to an S3 bucket, and an analyst's report reads from three of them. Nobody can answer "where is our PII?", "who owns this table?", or "if this column is wrong, what breaks downstream?" — because there is no map, no classification, and no catalog. Ungoverned data is un-securable data: you cannot protect, retain, or grant access to something you cannot see. This solution builds the map and the catalog so the estate becomes discoverable, classified, and owned.

The four building blocks

This solution stands up four things in order. Each is named once in the Implementation Guide and reused everywhere.

1. **The Data Map** — a live inventory of your data sources and the assets inside them. You register sources (Azure Blob/ADLS, Azure SQL, Synapse, AWS S3, GCP, on-premises SQL, and more) into a **collection** hierarchy that also carries access control. The Data Map is the foundation; nothing else works

without it.

1. **Scanning & classification** — scans connect to each registered source, enumerate its assets

(databases, tables, files), and apply **classifications** (system classifications such as credit card or national ID number, plus any custom classifications you define). Classification is what turns a raw inventory into a *sensitivity-aware* map.

1. **The Unified Catalog** — the business-facing layer built on top of the map: **governance

domains **(business groupings),** data products **(curated, published sets of assets with an owner and a use purpose), a** business glossary **(agreed terms and critical data elements), and** data quality**. This is where the estate becomes usable by the business, not just visible to IT.

1. **Lineage** — the wiring that shows how data flows from source to report. Lineage is captured

automatically from supported systems (Azure Data Factory, Synapse, Power BI, and more) and from scans, so you can trace an asset upstream to its origin and downstream to its consumers.

Collections, governance domains, and roles

Access in the Data Map is organized by **collections** — a hierarchy under the Purview account. Each collection carries its own role assignments: **Collection admins** (manage the collection and its roles), **Data source admins** (register sources and run scans), **Data curators** (edit catalog metadata and classifications), and **Data readers** (browse the catalog). The Unified Catalog adds **governance roles** (governance domain owners, data product owners, data stewards, data quality stewards). Least-privilege role assignment is a first-class step, not an afterthought — the Implementation Guide names each role at each stage.

Credentials and network — safe by design

Scanning reaches into live systems, so it needs credentials and network paths. This solution follows the safe-by-design pattern Microsoft documents: the Purview account's **managed identity** and an **Azure Key Vault** hold source credentials; **no secret is ever stored in a script or document**. Cloud sources are reached through the **Azure (managed) integration runtime**; on-premises and network-restricted sources are reached through a **self-hosted integration runtime (SHIR)** you install inside the network. Scans are always **piloted on a bounded scope first** before you scan the whole estate.

Licensing and the evolving portal

Microsoft Purview Data Governance (the Unified Catalog and Data Map) is billed through a **consumption / pay-as-you-go** model tied to an **Azure subscription**, and the capability set is evolving — some features (for example, **data quality** and certain **multicloud connectors**) may be in preview or may have changed name or availability. **Verify current pricing, availability, and portal paths for your tenant** in Microsoft's documentation and in the Purview portal before you plan. This solution cites the method; it never asserts a price or a portal path as permanent.

What this is not

This is the *data-estate governance* solution. It is **not** the Microsoft 365 compliance configuration (sensitivity/retention labels, DLP, insider risk, records management) — those are separate Purview

solutions in this category. It complements them: labels and DLP protect M365 content; the Data Map and Unified Catalog govern everything else and tie the picture together.

The deployment model

> *Assess the estate and readiness, design the collection/domain and role model, configure the Data Map and scans, **pilot scanning on a bounded scope**, validate the results against the gate, then expand across the estate — enforcing broader scanning and catalog curation only after the pilot passes. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog

Verify every requirement before you register a source or run a scan. Each entry gives Description, Why, How to verify, and How to implement if missing. Category-wide prerequisites (tenant, roles, toolchain, scope, stakeholders, pilot) are in `global-requirements-purview.md` — this guide adds the prerequisites specific to **Data Governance / Data Map / Unified Catalog**.

> ***Licensing/availability caveat.** Microsoft Purview Data Governance is billed by a consumption / pay-as-you-go model on an Azure subscription and is evolving; some features may be in preview. Where this guide names a capability, **verify its current availability and pricing for your tenant** in Microsoft's documentation and the Purview portal before relying on it.*

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription

Description: A Microsoft Purview account — the Data Map / Unified Catalog resource — provisioned in an **Azure subscription** (this is the data-governance resource, distinct from the M365 compliance portal). The account exposes the collection hierarchy, scanning, and the catalog. Why: The Data Map, scans, and Unified Catalog all live in this account; without it there is nothing to register sources into. How to verify: The account is visible in the Azure portal and its data-plane endpoint (`https://<account>.purview.azure.com`) resolves; run `PURVGOV-Discovery.ps1` to confirm reachability. How to implement if missing: Provision a Microsoft Purview account under change control on the target subscription; confirm the consumption billing model and region with your Azure owner.

2.2 Least-privilege data-governance roles (collections + Unified Catalog)

Description: Role assignments at the right scope — **Collection admin**, **Data source admin**, **Data curator**, **Data reader** on the Data Map collections, plus **governance domain owner / data product owner / data steward** in the Unified Catalog. Azure **Owner/Contributor** on the account only where account-level changes are required. Why: Purview uses fine-grained, collection-scoped roles; each configuration step requires the matching role, and least-privilege is a governance requirement in its own right. How to verify: In the Purview portal, open the collection's **Role assignments** and the Unified Catalog role pages; run `PURVGOV-Discovery.ps1` (reports collections and role coverage, read-only). How to implement if missing: Assign the least-privilege role at the narrowest collection/domain scope under change control; reserve account-level Collection admin for a small named group.

2.3 Data sources in scope, inventoried with owners

Description: A written list of the **data sources** to govern (for example Azure SQL, ADLS Gen2, Azure Blob, Synapse, Power BI, AWS S3, GCP BigQuery/Storage, on-premises SQL Server), each with a named **data owner** and **data steward**. Why: You register, scan, and catalog per source; classification and ownership are meaningless without a known owner per source. Availability of specific connectors (especially multicloud) may be preview — verify. How to verify: For each source you can name the system, its location/host, and its owner; run `PURVGOV-Set-Scope.ps1` to record the pilot source set. How to implement if missing: Inventory the estate with the data owners, confirm connector availability for each source type, and assign an owner and steward to every source before registering it.

2.4 Source credentials via managed identity / Key Vault — never in code

Description: A credential path for scanning each source: the Purview account's **managed identity** where the source supports it, otherwise a service principal or account secret stored in an **Azure Key Vault** linked to the Purview account. **No credential is ever placed in a script or document.** Why: Scans authenticate to live systems; storing secrets safely (Key Vault + managed identity) is mandatory and is how the provided scripts stay secret-free. How to verify: A Key Vault is linked to the Purview account and holds the source secrets, or the managed identity is granted read on the source; the scan's **Test connection** succeeds. How to implement if missing: Link a Key Vault to the account, grant the managed identity the least read role on each source (or store a secret in Key Vault), and verify **Test connection** before scanning.

2.5 Network path to each source (integration runtime)

Description: A working network path from the scanner to each source: the **Azure (managed) integration runtime** for reachable cloud sources, or a **self-hosted integration runtime (SHIR)** installed inside the network for on-premises and VNet/firewall-restricted sources. Why: A scan cannot classify what it cannot reach; on-premises and private-network sources require a SHIR with line of sight to the source. How to verify: The integration runtime shows **Running/Available** in the portal and **Test connection** on the source succeeds; `PURVGOV-Discovery.ps1` reports registered runtimes. How to implement if missing: Use the managed runtime for public cloud sources; install and register a SHIR on a host with network access for on-premises/private sources, then re-test the connection.

2.6 A classification approach (system + any custom classifications)

Description: A decision on how assets will be classified: rely on the **built-in system classifications** (credit card, national ID, and the large built-in set) and define any **custom classifications / classification rules** the organization needs (for example an internal customer-ID pattern). Why: Classification is what turns the raw Data Map into a sensitivity-aware inventory that drives ownership, access, and downstream protection. How to verify: The system classifications are visible on a test scan result; any custom classification rule is defined and matches sample data. How to implement if missing: Start with system classifications for the pilot; define custom classification rules only where a real internal pattern isn't covered, and validate each against sample data before estate-wide use.

2.7 Governance-domain and data-product intent (Unified Catalog)

Description: An agreed first **governance domain** (a business grouping such as Finance or Customer) and at least one intended **data product** (a curated, publishable set of assets with an owner and a use purpose), plus the initial **business glossary** terms / critical data elements for the pilot. Why: The Unified Catalog is organized by governance domains and data products; without an agreed first domain and product there is nothing to curate the pilot's scanned assets into. How to verify: Stakeholders have named the pilot governance domain, one data product, and a short initial glossary; `PURVGOV-Set-Scope.ps1` records them as the pilot scope. How to implement if missing: Facilitate a short workshop with the data owners to agree the pilot domain, one data product, its owner, and 5–10 initial glossary terms.

2.8 A pilot scope and change control

Description: A **bounded pilot** — one collection, one governance domain, and one or two sources — to prove registration, scanning, classification, and catalog curation before estate-wide rollout, plus a change-control window for scans (scans consume resources and touch live systems). Why: Every solution in this category follows Assess → Design → Configure → **Pilot** → Validate → Expand; scanning is resource- and access-sensitive and is always proven on a bounded scope first. How to verify: A named pilot collection/domain, its sources, and a change window exist; run `PURVGOV-Set-Scope.ps1` to materialize the pilot scope. How to implement if missing: Choose one well-understood, high-value domain and one or two of its sources as the pilot; agree the scan window with the source owners.

Checklist

- Purview account (Data Map) provisioned on an Azure subscription — `PURVGOV-Discovery.ps1`
- Least-privilege collection + Unified Catalog roles assigned — `PURVGOV-Discovery.ps1`
- Sources inventoried with owners/stewards — `PURVGOV-Set-Scope.ps1`
- Credentials via managed identity / Key Vault (no secrets in code)
- Network path per source (managed IR or SHIR) — Test connection passes
- Classification approach agreed (system + any custom)
- Pilot governance domain + data product + initial glossary agreed — `PURVGOV-Set-Scope.ps1`
- Pilot scope and scan change window agreed

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are*

*in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog

Follow these steps in order. This guide stands up the Data Map and roles, registers and scans a pilot source set, curates the Unified Catalog, wires lineage and data quality, then expands across the estate. Artifact names are fixed — create them with the exact names shown and reuse them everywhere. **Every scan is piloted on a bounded scope before the estate is scanned.**

> *Model: build the map, prove classification and curation on a pilot scope, then expand. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** Verify current portal paths and capability availability for your tenant as you go — the Purview Data Governance experience is evolving.*

Step 1 — Assess current state (read-only)

1. Run `PURVGOV-Discovery.ps1` to capture readiness signals: the Purview account endpoint is reachable, which **collections** exist and their **role coverage**, which **integration runtimes** are registered, and how many sources/assets are already known. No changes are made.

1. Record gaps against the Requirements checklist. Do **not** proceed past a missing account, missing least-privilege roles, or a missing network path — remediate those first.

Step 2 — Design the collection, domain, and role model

1. Decide the **collection hierarchy** under the account. For the pilot, create one collection named `DG-Collection-Pilot` under the root; plan the broader hierarchy (by business unit or by sensitivity) on paper for the expansion phase.

1. Decide the first **governance domain** named `DG-Domain-Pilot` (for example, Finance) in the Unified Catalog.

1. Assign least-privilege roles at the narrowest scope: **Collection admin** to a small named group on `DG-Collection-Pilot`, **Data source admin** to whoever will register/scan, **Data curator** to the catalog editors, **Data reader** to browsers; assign **governance domain owner** and **data product owner** in the Unified Catalog. Reserve account-level admin for a minimal named group.

Step 3 — Establish credentials and network paths (safe by design)

1. Link an **Azure Key Vault** to the Purview account (or confirm the account's **managed identity** can read each source). **Do not put any secret in a script or document.**

1. For each pilot source, grant the least read role to the managed identity or store the secret in Key Vault.

1. Confirm the **integration runtime** for each source: the **Azure (managed) runtime** for reachable cloud sources, or install and register a **self-hosted integration runtime (SHIR)** for on-premises or private-network sources. Confirm it shows **Running/Available**.

Step 4 — Register the pilot sources into the Data Map

1. Run `PURVGOV-Set-Scope.ps1` to record the pilot source set and the pilot domain/data-product intent

(read-only; produces the scope manifest you register against).

1. Register the pilot sources (for example `sql-fin-01.contoso.com` Azure SQL and an ADLS Gen2

container) into `DG-Collection-Pilot`. You may use the portal, or run `PURVGOV-Register-Source.ps1` which **defaults to** `-WhatIf` and prints exactly what it would register; re-run with `-Commit` to register.

1. Run **Test connection** for each registered source and confirm it passes before scanning.

Step 5 — Configure and pilot the scan (bounded scope, pilot-first)

1. Define the scan configuration: a **scan rule set** named `DG-ScanRuleSet-Pilot` (start from the

system default rule set), the **classifications** to apply (system classifications plus any custom rule named `DG-Classification-Custom-<name>`), and a **scoped** scan that targets only the pilot assets — not the whole source.

1. Trigger the scan **once** on the bounded scope (not scheduled yet). Use

`PURVGOV-Register-Source.ps1 -ConfigureScan` (`-WhatIf` by default; `-Commit` to apply) or the portal. Pass/fail: the scan completes with **status Succeeded** and returns assets with classifications.

1. **Rollback:** if a scan misbehaves or over-reaches, cancel the run in the portal and set the scan to

disabled; no source data is modified by a scan, but cancel to stop resource consumption. Re-scope and re-run.

Step 6 — Review scanned assets and classifications

1. Run `PURVGOV-Report.ps1` to list the pilot assets discovered, the classifications applied, and any

assets returned **unclassified**.

1. With the data owner, confirm the classifications are correct. Where a real internal pattern was

missed, define a **custom classification rule** (`DG-Classification-Custom-<name>`), validate it against sample data, add it to `DG-ScanRuleSet-Pilot`, and re-scan the pilot scope.

Step 7 — Curate the Unified Catalog (glossary, data product, ownership)

1. Create the initial **business glossary** term set named `DG-Glossary-Pilot` with the 5–10 agreed

terms / critical data elements; use `PURVGOV-Curate-Catalog.ps1 -Glossary` (`-WhatIf` by default; `-Commit` to write).

1. Create the first **data product** named `DG-DataProduct-Pilot` in `DG-Domain-Pilot`: add the

relevant scanned assets, set its **owner**, **use purpose**, and **terms of use**; use `PURVGOV-Curate-Catalog.ps1 -DataProduct` (`-WhatIf`/`-Commit`).

1. Assign **ownership and stewardship** on the assets and the data product so every published asset has

a named owner and steward.

Step 8 — Wire lineage

1. Confirm **lineage** is captured for the pilot's supported systems (for example, an Azure Data Factory or Synapse pipeline, or a Power BI dataset that reads the pilot source). Lineage populates automatically from these systems and from scans.

1. In the catalog, open a pilot asset and confirm you can trace it **upstream** to its source and **downstream** to its consumers. Record any gaps where a system is not a lineage source.

Step 9 — Establish data quality (verify availability)

1. If **data quality** is available in your tenant (this feature may be preview — verify), define a small **data-quality rule set** named `DG-DQ-Ruleset-Pilot` against the pilot data product's critical data elements (for example, completeness or format rules on a key column).

1. Run the data-quality assessment and record the baseline **quality score**. If the feature is not available in your tenant, record that and defer this step to the expansion phase.

Step 10 — Validate against the pilot gate

1. Complete the Testing Guide. The pilot passes only when: the pilot sources are registered and **Test connection** passes; the scan **Succeeded** and returned classified assets; the Unified Catalog has `DG-Glossary-Pilot` and `DG-DataProduct-Pilot` with owners assigned; and lineage resolves for at least one pilot asset.

1. Remediate any gaps and re-run the relevant validation before proceeding. Do **not** widen scope until the gate passes.

Step 11 — Expand across the estate (enforce)

1. After the pilot passes, add collections and register further sources one grouping at a time, re-running `PURVGOV-Set-Scope.ps1`, `PURVGOV-Register-Source.ps1`, and `PURVGOV-Report.ps1` for each new scope.

1. Move scans from one-time to **scheduled** only once a scope is validated, so classifications stay current as data changes.

1. Curate each newly scanned scope into governance domains and data products, extend `DG-Glossary-*`, and keep ownership assigned. Track coverage with `PURVGOV-Report.ps1`.

Order of operations

```
Assess -> Design (collections/domain/roles) -> Credentials & network ->
Register pilot sources -> Pilot scan (bounded) -> Review classifications ->
```

```
Curate catalog (glossary/data product/ownership) -> Wire lineage -> Data
quality -> Validate gate -> Expand (scheduled scans)
```

5. Testing Guide — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog

Run these tests after the pilot scan and catalog curation (Implementation Steps 5–9) and again after each expansion scope. Each test confirms one part of the governed estate works end to end. All read-only checks are safe to repeat.

Test 1 — Account and roles are ready

Do: Run `PURVGOV-Discovery.ps1`. Expected: The Purview account endpoint is reachable, `DG-Collection-Pilot` exists, and least-privilege roles (Collection admin, Data source admin, Data curator, Data reader) are reported as assigned; no required role is missing.

Test 2 — Pilot sources are registered and reachable

Do: Open `DG-Collection-Pilot` and run **Test connection** on each registered source; confirm with `PURVGOV-Report.ps1`. Expected: Each pilot source is registered and **Test connection** passes (managed IR or SHIR shows Running/Available). No source is unreachable.

Test 3 — The scan succeeds and returns assets

Do: Review the pilot scan run in the portal and run `PURVGOV-Report.ps1`. Expected: The scan status is **Succeeded**, the run scanned the intended bounded scope, and assets (databases/tables/files) were returned into the Data Map.

Test 4 — Assets are classified

Do: Run `PURVGOV-Report.ps1` and review the classification coverage for the pilot assets. Expected: System (and any custom) classifications are applied to the assets that contain matching data; the count of unexpectedly **unclassified** assets is understood and accepted by the data owner.

Test 5 — The business glossary exists

Do: Open the Unified Catalog glossary and confirm `DG-Glossary-Pilot`. Expected: The agreed 5–10 terms / critical data elements are present and, where intended, linked to assets.

Test 6 — The data product is published with an owner

Do: Open `DG-Domain-Pilot` and the data product `DG-DataProduct-Pilot`. Expected: The data product contains the intended assets, has a named **owner**, a **use purpose**, and **terms of use**; every published asset has an owner/steward.

Test 7 — Lineage resolves

Do: Open a pilot asset in the catalog and view its lineage. Expected: The asset traces **upstream** to its source and **downstream** to at least one consumer for a supported system; gaps (unsupported systems) are recorded, not silently missing.

Test 8 — Data quality baseline (if available)

Do: If data quality is available in your tenant (verify), run the `DG-DQ-Ruleset-Pilot` assessment. Expected: A baseline **quality score** is produced for the pilot data product's critical data elements. If the feature is unavailable, this test is recorded as deferred, not failed.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass: sources registered and reachable, scan Succeeded with classified assets, glossary and data product published with owners, and lineage resolves for at least one asset (Test 8 passes or is recorded as deferred where data quality is unavailable)
Expand pass	Tests 1–7 pass for each new scope, and scheduled scans are enabled only after the scope validates

6. Troubleshooting Guide — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog

Use `PURVGOV-Discovery.ps1` for readiness/role/runtime state, `PURVGOV-Report.ps1` for scan and classification coverage and catalog health, `PURVGOV-Set-Scope.ps1` to (re)record a scope, `PURVGOV-Register-Source.ps1` (`-WhatIf` by default) to register/scan, and `PURVGOV-Curate-Catalog.ps1` (`-WhatIf` by default) to curate. All scripts are read-only unless you pass `-Commit`.

A. Cannot reach the account / token fails

Symptom	Cause	Remediation
`Discovery` reports endpoint GAP	Wrong account name or endpoint	Confirm the account name; the endpoint is `https://<account>.purview.azure.com`; re-run `PURVGOV-Discovery.ps1`
Token acquisition fails	Not signed in / wrong resource	Run `Connect-AzAccount`; the scripts request a token for `https://purview.azure.net` — confirm your account can obtain it
401/403 on every call	Missing least-privilege role	Assign Data reader (browse) or the specific role the action needs at the collection scope; re-run

Symptom	Cause	Remediation
Calls fail with "api-version" errors	The example API version has moved	The Data Governance experience is evolving — verify and set the current data-plane API version for your tenant

B. Source registration or Test connection fails

Symptom	Cause	Remediation
Register returns 403	Not a Data source admin on the collection	Assign Data source admin on `DG-Collection-Pilot`; re-run `PURVGOV-Register-Source.ps1 -Commit`
Test connection fails (cloud source)	Managed identity lacks read on the source	Grant the account's managed identity the least read role on the source; re-test
Test connection fails (on-prem/private)	No network path	Install and register a self-hosted integration runtime (SHIR) with line of sight to the source; confirm Running; re-test
"Credential not found"	Key Vault credential name wrong or vault not linked	Link the Key Vault to the account and reference the credential **by name** (`-KeyVaultCredential`); never inline a secret
Unknown source "kind"	Connector name/availability differs	Verify the exact connector `kind` and its availability (some multicloud connectors are preview) in Microsoft's docs

C. Scan runs but classifies nothing

Symptom	Cause	Remediation
Assets returned, 0 classified	Scan rule set has no classifications	Confirm `DG-ScanRuleSet-Pilot` includes system classifications; re-scan and review with `PURVGOV-Report.ps1`
An internal pattern isn't detected	No custom classification rule	Define `DG-Classification-Custom-<name>`, validate against sample data, add to the rule set, re-scan
Scan scope too broad / slow / costly	Whole source scanned instead of pilot scope	Re-scope the scan to the bounded pilot assets; keep it one-time until validated

Symptom	Cause	Remediation
Scan status Failed	Transient/auth/network	Open the run in the portal for the failure reason; fix credential/runtime; re-run

D. Classifications look wrong

- False positives/negatives are expected on first scan. Review with the **data owner** using

`PURVGOV-Report.ps1`; tune or remove a custom classification rule and re-scan the pilot scope. The solution provides the technical result — the owner decides whether a classification is correct for the business.

E. Unified Catalog curation issues

Symptom	Cause	Remediation
Glossary/data-product create returns 403	Not a Data curator / governance domain owner	Assign the Unified Catalog role for the domain; re-run `PURVGOV-Curate-Catalog.ps1 -Commit`
Data product has no owner	Owner not set	Set `-Owner`; every published data product and asset must have a named owner/steward
Curation API path 404 / version error	Unified Catalog API evolving	Verify the current Unified Catalog / Atlas glossary and data-product API versions for your tenant

F. Lineage is missing

- Lineage populates automatically from **supported** systems (Azure Data Factory, Synapse, Power BI,

and more) and from scans. If a pilot asset shows no lineage, confirm the producing system is a supported lineage source and is connected to Purview; record unsupported systems as known gaps rather than expecting lineage from them.

G. Data quality unavailable

- Data quality may be in **preview** or not enabled for your tenant. If `DG-DQ-Ruleset-Pilot` cannot

be created, verify the feature's availability and licensing for your tenant; record the step as deferred to the expansion phase — it is not a pilot failure.

H. Diagnostics quick reference

```
.\PURVGOV-Discovery.ps1 -PurviewAccount contoso-purview           # readiness: endpoint, collections, rol
.\PURVGOV-Set-Scope.ps1 -Source @('sql-fin-01.contoso.com/FinanceDB')  # record a pilot scope manifest
.\PURVGOV-Register-Source.ps1 -PurviewAccount contoso-purview -Collection DG-Collection-Pilot `
    -SourceName sql-fin-01 -Kind AzureSqlDatabase                # -WhatIf by default; add -Commit to app
.\PURVGOV-Report.ps1 -PurviewAccount contoso-purview -Collection DG-Collection-Pilot  # scan + classificat
```

```
.\PURVGOV-Curate-Catalog.ps1 -PurviewAccount contoso-purview -Glossary -TermName "Customer ID"  # -WhatIf
```

I. When to escalate

If the account is reachable, roles are assigned, the runtime is Running, Test connection passes, and a scoped scan still returns no classified assets after tuning the rule set, capture the `PURVGOV-Discovery.ps1` and `PURVGOV-Report.ps1` output and the portal scan-run detail, and review the source connectivity and rule set with the data owner and your Purview/Azure administrator. For evolving or preview features, verify current availability and API versions against Microsoft's documentation before assuming a defect.

7. Best Practices — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog

• Design the collection hierarchy before you register anything. Collections carry both organization

and access control; a hierarchy planned by business unit or sensitivity (with `DG-Collection-Pilot` as the first node) is far cheaper than re-parenting sources later.

• Least-privilege by collection, always. Assign Data source admin, Data curator, and Data reader at

the narrowest collection scope; reserve account-level Collection admin for a small named group. Broad roles at the root are the most common governance finding.

• Never put a secret in a script or scan. Use the account's managed identity or an Azure Key Vault

linked to the account for every source credential — this is both the secure pattern and what keeps the provided scripts secret-free.

• Pilot every scan on a bounded scope. Scoped, one-time scans on a pilot source set prove

classification and cost before you point a scan at an entire lake or database estate; move to scheduled scans only after a scope validates.

• Trust system classifications first, add custom sparingly. The built-in classifications cover most

regulated data; define a custom classification rule (`DG-Classification-Custom-<name>`) only for a real internal pattern, and validate it against sample data before estate-wide use.

• Curate to data products, not just assets. A raw classified asset is visible to IT; a published

data product with an owner, a use purpose, and terms of use is what the business can actually find and trust — curation is the point of the catalog.

• Every published asset has an owner and a steward. Ownership is what makes classification and access

decisions accountable; an unowned asset in the catalog is a governance gap, not a governed asset.

• Let lineage come from the source systems. Lineage populates automatically from supported systems

(Data Factory, Synapse, Power BI); wire those integrations rather than hand-drawing lineage, and record which systems are not lineage sources so the gaps are known.

• Treat determinations as someone else's job. The solution classifies, maps, and catalogs; the

business/records/legal owners decide sensitivity, retention, and access. Keep that separation explicit so technical scanning is never mistaken for a compliance ruling.

- Verify licensing and portal paths as you go. The Data Governance experience, its feature set (data quality, multicloud connectors), and its consumption billing are evolving — confirm current availability and pricing in Microsoft's docs and the portal rather than assuming a path or a price is permanent.

8. Deliverables Summary — Microsoft Purview Data Governance: Data Map, Scanning & Unified Catalog

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The Data Map, scanning + classification, the Unified Catalog (domains, data products, glossary, data quality), lineage, and how it complements M365 Purview (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: design, register, pilot-scan, curate, wire lineage, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Pilot collection	`DG-Collection-Pilot`
Pilot governance domain	`DG-Domain-Pilot`
Scan rule set	`DG-ScanRuleSet-Pilot`
Custom classification rule (as needed)	`DG-Classification-Custom-<name>`
Business glossary term set	`DG-Glossary-Pilot`
Data product	`DG-DataProduct-Pilot`
Data-quality rule set (if available)	`DG-DQ-Ruleset-Pilot`

PowerShell scripts

Script	What it does
`PURVGOV-Discovery.ps1`	Reports data-governance readiness: account endpoint reachable, collections + role coverage, registered integration runtimes, and known source/asset counts (Purview data-plane REST via an Azure token; read-only)
`PURVGOV-Register-Source.ps1`	Registers a data source and configures a scoped scan; **defaults to** `-WhatIf` (prints the plan), applies only with `-Commit`; no secrets in code
`PURVGOV-Curate-Catalog.ps1`	Unified Catalog curation helper for a glossary term, a classification association, or a data product; `-WhatIf` **by default**, writes only with `-Commit`
`PURVGOV-Report.ps1`	Scan + classification coverage and catalog-health report: scan run status, classified vs unclassified assets, glossary/data-product/ownership coverage (read-only)
`PURVGOV-Set-Scope.ps1`	Records the pilot/expansion scope — the source set, governance domain, and data-product intent — as a scope manifest to configure against (read-only)

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

A governed data estate — sources mapped, assets scanned and classified, the Unified Catalog curated with domains, data products, glossary, and owners, and lineage wired — discoverable and governed far beyond Microsoft 365, proven on a pilot scope before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PURVGOV-Curate-Catalog.ps1

Purpose: Curates the Unified Catalog for the pilot: create a business-glossary term (-Glossary), associate a classification with an asset (-Classify), or create a data product with an owner (-DataProduct). SAFE BY DEFAULT: with no -Commit it runs in -WhatIf mode and prints the exact payload it WOULD send, changing nothing. Pass -Commit to write. No secrets in code. HOW TO RUN Glossary term (preview by default): .\PURVGOV-Curate-Catalog.ps1 -PurviewAccount contoso-purview -Glossary -TermName "Customer ID" `-TermDefinition "The unique enterprise identifier for a customer." Data product (apply): .\PURVGOV-Curate-Catalog.ps1 -PurviewAccount contoso-purview -DataProduct` -DataProductName DG-DataProduct-Pilot -Domain DG-Domain-Pilot -Owner "owner@contoso.com" -Commit Requires Az.Accounts + Connect-AzAccount with Data curator / governance domain owner as appropriate. EXPECTED RESULTS Default: a printed "[WhatIf] would create/associate ..." plan and NO change. With -Commit the catalog object is created/updated and the script prints its name/GUID. Verify the current Unified Catalog / Atlas glossary API versions for your tenant — the experience is evolving. .NOTES Purview — Data Governance. Writes gated behind -Commit; -WhatIf default. Determinations (what a term MEANS, who OWNS an asset) are the data owners' — this tool records their decisions, not makes them. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [switch]$Glossary, [switch]$Classify, [switch]$DataProduct, glossary [string]$TermName, [string]$TermDefinition, [string]$GlossaryName = 'DG-Glossary-Pilot', classify [string]$AssetGuid, [string]$Classification, # e.g. MICROSOFT.GOVERNMENT.US.SOCIAL_SECURITY_NUMBER or a custom name data product [string]$DataProductName = 'DG-DataProduct-Pilot', [string]$Domain = 'DG-Domain-Pilot', [string]$Owner, [string]$UsePurpose = 'Pilot data product for governed discovery.', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

How to run: Glossary term (preview by default): .\PURVGOV-Curate-Catalog.ps1 -PurviewAccount contoso-purview -Glossary -TermName "Customer ID" `-TermDefinition "The unique enterprise identifier for a customer." Data product (apply): .\PURVGOV-Curate-Catalog.ps1 -PurviewAccount contoso-purview -DataProduct` -DataProductName DG-DataProduct-Pilot -Domain DG-Domain-Pilot -Owner "owner@contoso.com" -Commit Requires Az.Accounts + Connect-AzAccount with Data curator / governance domain owner as appropriate. EXPECTED RESULTS Default: a printed "[WhatIf] would create/associate ..." plan and NO change. With -Commit the catalog object is created/updated and the script prints its name/GUID. Verify the current Unified Catalog / Atlas glossary API versions for your tenant — the experience is evolving. .NOTES Purview — Data Governance. Writes gated behind -Commit; -WhatIf default. Determinations (what a term MEANS, who OWNS an asset) are the data owners' — this tool records their decisions, not makes them. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [switch]$Glossary, [switch]$Classify, [switch]$DataProduct, glossary [string]$TermName, [string]$TermDefinition, [string]$GlossaryName = 'DG-Glossary-Pilot', classify [string]$AssetGuid, [string]$Classification, # e.g. MICROSOFT.GOVERNMENT.US.SOCIAL_SECURITY_NUMBER or a custom name data product [string]$DataProductName = 'DG-DataProduct-Pilot', [string]$Domain = 'DG-Domain-Pilot', [string]$Owner, [string]$UsePurpose = 'Pilot data product for governed discovery.', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

PURVGOV-Discovery.ps1

Purpose: Reports data-governance readiness so you can assess current state before registering sources or scanning: whether the Microsoft Purview account data-plane endpoint is reachable, which collections exist and their role-assignment coverage, which integration runtimes are registered, and a high-level count of registered data sources. Read-only — it makes no change to the account, sources, or catalog. Supports the Assess step. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVGOV-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes, and registered data-source count — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — Data Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\PURVGOV-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes, and registered data-source count — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — Data Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

PURVGOV-Register-Source.ps1

Purpose: Registers a data source into a Data Map collection and (with -ConfigureScan) prepares a scoped, one-time scan using a named scan rule set. SAFE BY DEFAULT: with no -Commit it runs in -WhatIf mode and only prints the exact registration/scan payload it WOULD send — it changes nothing. Pass -Commit to apply. No credential is ever placed in this script; source authentication uses the account's managed identity or an Azure Key Vault credential referenced by name. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\PURVGOV-Register-Source.ps1 -PurviewAccount contoso-purview `-Collection DG-Collection-Pilot -SourceName sql-fin-01 -Kind AzureSqlDatabase` -ServerEndpoint sql-fin-01.contoso.com -Database FinanceDB Apply: ... same args ... -ConfigureScan -ScanRuleSet DG-ScanRuleSet-Pilot -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a scan definition is created; the script prints the resulting object names. Trigger/monitor the scan run in the portal or with PURVGOV-Report.ps1. .NOTES Purview — Data Governance. Writes gated behind -Commit; -WhatIf is the default. Verify current Purview scan API versions and the exact source "kind" for your connector in Microsoft's docs. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory =

$true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$Kind, # e.g. AzureSqlDatabase, AdlsGen2, AmazonS3 — verify per connector [string]$ServerEndpoint, [string]$Database, [switch]$ConfigureScan, [string]$ScanRuleSet = 'DG-ScanRuleSet-Pilot', [string]$KeyVaultCredential, # NAME of a Key Vault credential already linked to the account [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\PURVGOV-Register-Source.ps1 -PurviewAccount contoso-purview `-Collection DG-Collection-Pilot -SourceName sql-fin-01 -Kind AzureSqlDatabase` -ServerEndpoint sql-fin-01.contoso.com -Database FinanceDB Apply: ... same args ... -ConfigureScan -ScanRuleSet DG-ScanRuleSet-Pilot -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a scan definition is created; the script prints the resulting object names. Trigger/monitor the scan run in the portal or with PURVGOV-Report.ps1. .NOTES Purview — Data Governance. Writes gated behind -Commit; -WhatIf is the default. Verify current Purview scan API versions and the exact source "kind" for your connector in Microsoft's docs. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$Kind, # e.g. AzureSqlDatabase, AdlsGen2, AmazonS3 — verify per connector [string]$ServerEndpoint, [string]$Database, [switch]$ConfigureScan, [string]$ScanRuleSet = 'DG-ScanRuleSet-Pilot', [string]$KeyVaultCredential, # NAME of a Key Vault credential already linked to the account [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

PURVGOV-Report.ps1

Purpose: Reports the health of the governed estate for a scope: latest scan run status per source, counts of classified vs unclassified assets, and Unified Catalog coverage (assets with owners, glossary terms defined, data products published). Read-only — it queries the Purview data-plane REST / catalog search and changes nothing. Use it to review the pilot (Testing Guide) and to track coverage as the estate expands. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVGOV-Report.ps1 -PurviewAccount contoso-purview ` -Collection DG-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A coverage summary: per-source last scan status (Succeeded/Failed/Running), classified vs unclassified asset counts with a classification-coverage %, and catalog-health lines (owners assigned, glossary terms, data products). PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — Data Governance. Read-only. Verify current search/scan API versions for your tenant; the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\PURVGOV-Report.ps1 -PurviewAccount contoso-purview ` -Collection DG-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A coverage summary: per-source last scan status (Succeeded/Failed/Running), classified vs unclassified asset counts with a classification-coverage %, and catalog-health lines (owners assigned, glossary terms, data products). PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — Data Governance. Read-only. Verify current search/scan API versions for your tenant; the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection,

[string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

PURVGOV-Set-Scope.ps1

Purpose: Captures and writes a scope manifest for the pilot (or an expansion wave): the target collection, the governance domain, the data-source set to register, and the intended data product and initial glossary terms. Read-only against the tenant — it writes only a local JSON manifest you register and curate against; it does not change the Purview account. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVGOV-Set-Scope.ps1 -Collection DG-Collection-Pilot `-Domain DG-Domain-Pilot -DataProduct DG-DataProduct-Pilot` -Source @('sql-fin-01.contoso.com/FinanceDB','adls-fin/finance-extracts') ` -GlossaryTerm @('Customer ID','Invoice Amount') -OutFile .\pilot-scope.json EXPECTED RESULTS A pilot-scope.json manifest listing the collection, governance domain, sources, data product, and glossary terms, plus a printed summary — used as the input to PURVGOV-Register-Source.ps1 and PURVGOV-Curate-Catalog.ps1. .NOTES Purview — Data Governance. Local manifest only; no tenant change. Names are white-label placeholders. > [CmdletBinding()] param([string]$Collection = 'DG-Collection-Pilot', [string]$Domain = 'DG-Domain-Pilot', [string]$DataProduct = 'DG-DataProduct-Pilot', [string[]]$Source = @('sql-fin-01.contoso.com/FinanceDB'), [string[]]$GlossaryTerm = @('Customer ID','Invoice Amount'), [string]$ScanRuleSet = 'DG-ScanRuleSet-Pilot', [string]$OutFile = '.\pilot-scope.json')

How to run: powershell -ExecutionPolicy Bypass -File .\PURVGOV-Set-Scope.ps1 -Collection DG-Collection-Pilot `-Domain DG-Domain-Pilot -DataProduct DG-DataProduct-Pilot` -Source @('sql-fin-01.contoso.com/FinanceDB','adls-fin/finance-extracts') ` -GlossaryTerm @('Customer ID','Invoice Amount') -OutFile .\pilot-scope.json EXPECTED RESULTS A pilot-scope.json manifest listing the collection, governance domain, sources, data product, and glossary terms, plus a printed summary — used as the input to PURVGOV-Register-Source.ps1 and PURVGOV-Curate-Catalog.ps1. .NOTES Purview — Data Governance. Local manifest only; no tenant change. Names are white-label placeholders. > [CmdletBinding()] param([string]$Collection = 'DG-Collection-Pilot', [string]$Domain = 'DG-Domain-Pilot', [string]$DataProduct = 'DG-DataProduct-Pilot', [string[]]$Source = @('sql-fin-01.contoso.com/FinanceDB'), [string[]]$GlossaryTerm = @('Customer ID','Invoice Amount'), [string]$ScanRuleSet = 'DG-ScanRuleSet-Pilot', [string]$OutFile = '.\pilot-scope.json')